

Guía Paso a Paso: Día 2 - Conexión con el Mundo Exterior (API)

¡Bienvenidos al Día 2! Hoy dejaremos de usar datos estáticos y conectaremos **MovieNexus** con el universo real de las películas. Pero antes de escribir una sola línea de código, vamos a entender el "por qué" de cada pieza del rompecabezas.

Contexto Pedagógico: La Comunicación Cliente-Servidor

En una aplicación profesional, el **Frontend** (Angular) no tiene las películas guardadas; le pertenecen al **Backend** (TMDB). Para obtenerlas, usamos el protocolo **HTTP**.

¿Qué vamos a construir hoy?

1. **Un Motor de Peticiones (HttpClient):** El vehículo para viajar por la red.
2. **Un Contrato de Datos (Interfaces):** El lenguaje común para que TypeScript entienda qué responde el servidor.
3. **Un Escudo Inteligente (Interceptor):** Para no repetir código de seguridad en cada rincón de la app.
4. **Un Mensajero Especializado (Service):** El encargado único de ir y volver de la API.

Paso 1: Configurar el Motor HTTP (*app.config.ts*)

¿Qué se va a hacer?

Registrar el proveedor de HttpClient en el corazón de la aplicación.

¿Por qué lo hacemos?

Angular es modular y eficiente. No incluye las herramientas de HTTP por defecto para que la aplicación pese menos. Si queremos salir a internet, debemos "pedir permiso" y habilitar esta funcionalidad explícitamente.

¿Cómo lo hacemos?

Modificamos *src/app/app.config.ts*:

```
import { ApplicationConfig } from '@angular/core';
import { provideHttpClient, withFetch, withInterceptors } from
'@angular/common/http';
import { apiInterceptor } from '../core/interceptors/api.interceptor';

export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient(
      withFetch(), // Habilita el uso de la API Fetch moderna (más
rápida y compatible con SSR)
      withInterceptors([apiInterceptor]) // Registra nuestro
interceptor (Paso 3)
    )
  ]
};
```

Paso 2: El Contrato de Datos (movie.model.ts)

¿Qué se va a hacer?

Crear interfaces de TypeScript que describan la forma de los datos de TMDB.

¿Por qué lo hacemos?

Sin interfaces, los datos serían de tipo any (desconocido). Esto es peligroso porque si intentamos leer `pelicula.titulo` pero la API lo llama `pelicula.title`, la app fallará. Las interfaces nos dan Autocompletado y Detección de errores en tiempo real.

¿Cómo lo hacemos?

Crea **`src/app/core/models/movie.model.ts`**:

```
export interface Movie {
  id: number;
  title: string;
  overview: string;
  poster_path: string;
  backdrop_path: string;
  vote_average: number;
  release_date: string;
}

export interface MovieResponse {
  results: Movie[]; // La API nos devuelve una lista de películas
  page: number;
  total_pages: number;
}
```

Paso 3: El Interceptor Funcional (api.interceptor.ts)

¿Qué se va a hacer?

Crear una función que "atrape" todas las peticiones salientes y les pegue la api_key.

¿Por qué lo hacemos? (Principio DRY - Don't Repeat Yourself)

Imagina que tienes 50 servicios distintos pidiendo datos Si pones la api_key manualmente en cada uno:

1. Perderías mucho tiempo.
2. Si la clave cambia, tendrías que editar 50 archivos.
3. El código se vería sucio. El interceptor es como un peaje automático: cada coche (petición) que pasa recibe su ticket (token/key) sin detenerse.

¿Cómo lo hacemos?

Crea **src/app/core/interceptors/api.interceptor.ts**:

```
import { HttpInterceptorFn } from '@angular/common/http';
import { environment } from '../../../environments/environment';

export const apiInterceptor: HttpInterceptorFn = (req, next) => {
  // 1. Verificamos si la petición va dirigida a TMDB
  if (req.url.includes('api.themoviedb.org')) {
    // 2. Clonamos la petición (son inmutables) y le añadimos
    // parámetros
    const apiReq = req.clone({
      setParams: {
        api_key: environment.apiKey,
        language: 'es-ES' // Configuramos el idioma globalmente
      }
    });
    return next(apiReq); // 3. Enviamos la petición modificada
  }
  return next(req); // Si no es de TMDB, la dejamos pasar normal
};
```

Paso 4: El Servicio Especializado (movie.service.ts)

¿Qué se va a hacer?

Crear una clase que se encargue exclusivamente de pedir películas.

¿Por qué lo hacemos? (Single Responsibility Principle)

Los componentes (como Home) deben ocuparse de mostrar cosas, no de saber de dónde vienen. Si el día de mañana cambiamos de TMDB a Netflix API, solo editamos el servicio, y el componente ni se entera.

¿Cómo lo hacemos?

Crea **src/app/core/services/movie.service.ts**:

```
import { HttpClient } from '@angular/common/http';
import { inject, Injectable } from '@angular/core';
import { environment } from '../../../environments/environment';
import { MovieResponse } from '../../../models/movie.model';

@Injectable({ providedIn: 'root' }) // Disponible en toda la app
export class MovieService {
  private http = inject(HttpClient); // Inyectamos el motor HTTP
  private apiUrl = environment.baseUrl;

  getTrendingMovies() {
    // Retornamos un Observable (una promesa de que llegarán datos)
    return this.http.get<MovieResponse>(
      `${this.apiUrl}/trending/movie/day`
    );
  }
}
```

Prueba de Aprendizaje

Para saber si realmente entendieron, los aprendices deben poder responder:

1. ¿Qué pasa si borro el Interceptor? (Respuesta: Las peticiones fallarán con error 401 Unauthorized).
2. ¿Por qué usamos req.clone() en lugar de modificar req directamente? (Respuesta: Las peticiones en Angular son inmutables por seguridad).
3. ¿Dónde puedo ver si la api_key se está enviando? (Respuesta: En la pestaña "Network" o "Red" del navegador, haciendo clic en la petición y mirando los "Payload" o "Query Parameters").

Reto Final:

inyecten el servicio en home.ts, llamen a `getTrendingMovies()` y hagan un `console.log` del resultado. Si ven un array de 20 películas... ¡Han dominado la conexión de datos!

El servicio lo deben inyectar en:

Archivo **`src/app/features/home/home.ts`**:

Verificación en consola del navegador

The screenshot shows a web browser at `localhost:4200` displaying the MovieNexus application. The application has a dark blue header with the text "Bienvenido" and "a MovieNexus". Below this, it says "Explora las películas más populares, busca tus favoritas y mantente al día con los últimos estrenos." and a button labeled "Empezar a Explorar".

The browser's developer console is open, showing the following messages:

- Angular is running in development mode.
- Home Inicializado. Cargando películas...
- ¡Éxito! Datos recibidos de TMDB: `Array(20)`
- Angular hydrated 4 component(s) and 40 node(s), 0 component(s) <https://v21.angular.dev/guide/hydration>.
- ERROR RuntimeError: NG04002: Cannot match any routes. URL Se...
- ERROR RuntimeError: NG04002: Cannot match any routes. URL Se...
- ERROR RuntimeError: NG04002: Cannot match any routes. URL S...

The console also shows a search bar at the bottom with the text "Buscar".

Paso Final: Control de Versiones (Git)

¿Qué se va a hacer?

Guardar los cambios del día en nuestro repositorio Git.

¿Por qué lo hacemos?

Como desarrolladores profesionales, nunca terminamos una tarea sin asegurar nuestro trabajo. El commit es nuestra "partida guardada". Si mañana algo falla, siempre podemos volver a este punto exacto donde todo funcionaba perfectamente.

¿Cómo lo hacemos?

Ejecuta estos comandos en tu terminal:

```
git add .  
git commit -m "feat: implementar conexión a TMDB con interceptor y  
servicio de películas"  
git push origin main
```